

# FortexAI

## A Multi-Layer Defense Framework Against Prompt Injection Attacks

MSc Data Science — Model Application Development

Project Stream: Data Privacy and Security

*Project Overview & Technical Documentation — Updated July 2026*

**Group No: 3 | Members: Ashir Sable (86092500029), Jivesh Gawde (86092500059), Mahat Vasudev (86092500008)**

*Prompt injection is ranked the #1 risk for LLM applications — and neither retrieval-augmented generation nor fine-tuning fully solves it. — Paraphrased from OWASP's LLM01:2025 Prompt Injection risk page*  
[\[genai.owasp.org\]](https://genai.owasp.org)

## Executive Summary

---

Every AI assistant, chatbot, and autonomous agent shares one unsolved weakness: it cannot always tell the difference between an instruction from its owner and text supplied by an attacker. This is prompt injection — ranked LLM01, the #1 risk on OWASP's official Top 10 for LLM Applications. It has already caused real breaches: a zero-click Microsoft 365 Copilot flaw that could exfiltrate enterprise data from a single crafted email, a compromised open-source library pulled roughly 47,000 times before removal, and AI shopping agents tricked into fraudulent crypto payments.

FortexAI is a self-hosted, layered defense system that screens prompts before they reach an LLM and screens responses before they reach the user. Instead of running one expensive check on every request, it uses a cost-optimized cascade — cheap, fast checks first, and an LLM "judge" only for genuinely ambiguous cases — a pattern validated by Stanford's FrugalGPT research, which showed cascades can match top-tier LLM performance at a fraction of the cost. A cosine-similarity output check catches leaks that slip past every input check, and every confirmed attack — and every confirmed-safe prompt — is fed back into a shared vector database, so the system gets smarter with every request it sees.

The same protection now extends beyond typed text: uploaded images and documents are run through OCR text extraction, and the extracted text is forwarded into the exact same cascade used for typed prompts, so an attack hidden as visible text inside a picture is caught the same way a typed attack would be. A separate image classifier was deliberately not built for this release — Section 3.4 explains why OCR-only coverage was judged the better trade-off. Fully invisible, pixel-level (steganographic) image attacks remain a separate, harder research problem that even frontier AI labs have not solved — this is discussed openly in Section 3.4 as a scoped decision, not an oversight.

The market has already priced this problem: in September 2025, Check Point paid an estimated \$300 million to acquire the AI-security startup Lakera — one of at least six similar acquisitions by major cybersecurity vendors within weeks of each other. FortexAI's opportunity is not to out-spend that consolidation, but to offer what none of the acquired platforms do: a free, self-hosted, fully transparent, output-aware layer that keeps every prompt inside the user's own infrastructure.

## 1. The Problem: AI Systems Cannot Tell Instructions From Attacks

---

Large language models process everything as text in a single stream — the system's instructions, the user's message, retrieved documents, and tool outputs all sit in the same context window with no hard boundary between them. An attacker who can place text anywhere the model will read it can potentially override what the model was told to do. This flaw is architectural, not a bug in any one product, which is why researchers increasingly describe it as a permanent property of how today's LLMs work rather than something a future patch will fix.

### 1.1 The numbers

Every figure below was checked against its original publisher, not a secondary blog repeating it. A few widely-circulated numbers (e.g. a claimed "340% year-over-year surge" in prompt injection attempts) could not be traced to any verifiable primary source and are deliberately excluded from this document.

| Metric                    | Finding                                                                                                                                                                   |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| #1 on OWASP LLM Top 10    | Prompt injection is LLM01:2025 — ranked the top risk on OWASP's official Top 10 for LLM Applications, maintained by the OWASP GenAI Security Project.                     |
| \$300M                    | Check Point's estimated acquisition price for AI-security startup Lakera, announced Sept 2025 — one signal of how much enterprises will pay to solve this.                |
| \$670K higher breach cost | IBM's 2026 Cost of a Data Breach Report studied 600 breached organizations; breaches linked to unauthorized ("shadow") AI use cost \$670,000 more than standard breaches. |
| 65% vs 53% PII exposure   | In the same IBM study, breaches linked to shadow AI exposed customer PII in 65% of cases, versus 53% across all breaches studied.                                         |

### 1.2 It is already happening

- June 2025 — "EchoLeak" (CVE-2025-32711, CVSS 9.3): a zero-click prompt injection flaw in Microsoft 365 Copilot let an attacker exfiltrate enterprise data from Word, Excel, Outlook, and Teams via a single crafted email — no user interaction required. Disclosed by Aim Security; patched server-side by Microsoft with no confirmed in-the-wild exploitation. [Source: [The Hacker News](#)]
- February 2026 — The "Clinejection" incident: a permission-bypass flaw in an AI coding tool's automated issue-triage workflow, disclosed Feb 9 and exploited in the wild within eight days via prompt injection hidden in GitHub issue titles. [Source: [Cloud Security Alliance](#)]
- March 2026 — An autonomous bot exploited a misconfigured CI pipeline and pushed backdoored versions of LiteLLM — a gateway library underlying CrewAI, DSPy, and Microsoft GraphRAG — to PyPI. Live for roughly three hours; downloaded close to 47,000 times, per the OWASP GenAI Security Project's incident reporting. [Source: [Help Net Security](#)]
- July 2026 — Security researchers documented threat actors using SEO-poisoned websites with hidden prompts to trick AI agents into making real cryptocurrency payments during routine "acquire an API key" workflows. [Source: [SecurityWeek](#)]

*An AI agent that can read private data, be exposed to content an attacker controls, and communicate externally is a target: an attacker who controls any of that untrusted content can trick the agent into reading the private data and sending it back to them.* — Paraphrased from Simon Willison's description of the "lethal trifecta" for AI agents [\[simonwillison.net\]](#)

This matters far beyond any one industry. Any organization deploying an AI assistant that reads external content (emails, documents, search results, support tickets) and can take an action (send data, make a payment, run code) is exposed by construction — not by misconfiguration.

## 2. Why Today's Guardrail Tools Leave Gaps

---

The market has responded, but mostly by consolidation rather than by closing the technical gaps. Within weeks of each other in 2025, Check Point acquired Lakera (~\$300M), Cato Networks acquired Aim Security, SentinelOne acquired Prompt Security, and Tenable acquired Apex — [Source: [calcalistech.com](https://calcalistech.com)]

F5 also acquired CalypsoAI and Varonis acquired SlashNext around the same time, while CrowdStrike announced its acquisition of Pangea the same day Check Point announced Lakera — six major cybersecurity vendors buying their way into AI security almost simultaneously. [Source: [CyberScoop](https://cyberscoop.com) / [SecurityWeek](https://securityweek.com)]

That is strong validation that the problem is real and valuable. It does not mean the resulting products solve it completely. Looking closely at what is actually available today reveals a consistent pattern: every option makes a trade-off that leaves a real gap open.

- Open-source scanning libraries (e.g. LLM Guard, MIT-licensed, 15 input + 20 output scanners as of its current release) are free and self-hosted, but ship as a library you wire in yourself — not a monitored product with a dashboard or an evolving attack memory. [Source: [appsec.santa.com](https://appsec.santa.com)]
- Self-hardening approaches (e.g. Rebuff) that remember past attacks are a good idea in principle, but independent practitioner comparisons flag maintenance status as a real concern — you inherit the burden of keeping the attack corpus current. [Source: [bestllm-scanners.com](https://bestllm-scanners.com)]
- Dialogue-policy frameworks (e.g. NVIDIA NeMo Guardrails) are strong at constraining what a chatbot may discuss, but are an orchestration layer, not a dedicated injection classifier, and practitioners report the in-process design hits limits once an organization's AI footprint grows past a single application. [Source: [deepinspect.ai](https://deepinspect.ai)]
- Managed cloud APIs (e.g. Lakera Guard, now part of Check Point) offer strong, fast, professionally maintained detection — Lakera's Gandalf red-teaming initiative has generated roughly 80 million attack patterns — but every prompt leaves the customer's network for evaluation, and the tool is explicitly scoped to input classification, not output-side hallucination or leak checking. [Source: [calcalistech.com](https://calcalistech.com)]
- Single free classifiers (e.g. Meta's Llama Prompt Guard) give you one useful signal, cheaply, but only one signal — no fusion with other detection methods, no dashboard, no output-side check.

The pattern is structural: the industry has largely built input-side classifiers of one kind or another. Almost nothing available today checks whether an attack actually succeeded by examining what the model said back — and nothing in this landscape combines cost-optimized staged detection with a self-improving shared memory in one open, self-hosted package.

### 3. Our Solution: FortexAI

---

FortexAI is built around a simple insight validated by real research: most traffic is obviously fine, a smaller fraction is obviously malicious, and only a small remainder is genuinely ambiguous. Rather than run every request through every possible check, FortexAI uses a cascade that escalates cost only when the evidence demands it. Stanford's FrugalGPT research (Chen, Zaharia & Zou, 2023) demonstrated that this style of cascade can match the accuracy of always using the most expensive model, at up to 98% lower cost. [Source: [arXiv:2305.05176](https://arxiv.org/abs/2305.05176)]

#### 3.1 The five stages

| Stage                        | What it does                                                                                                                                                                                                            | Why this technique                                                                                                                                                                                                                                |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Autoencoder               | Learns what a "normal" prompt looks like from ordinary usage logs; flags anything unusual as worth a closer look.                                                                                                       | Unsupervised — needs no labeled attack data, and clean/normal prompts skip straight through, keeping the system fast and cheap for the majority of traffic.                                                                                       |
| 2. Semantic search           | Compares the prompt's meaning (via embeddings) against two vector databases: previously confirmed attacks, and previously confirmed-normal ("allowed") prompts.                                                         | Catches paraphrased repeats of known attacks that keyword rules would miss, and lets normal-but-unusual traffic that the autoencoder flagged exit early once it has been confirmed benign before — without needing a full model inference call.   |
| 3. Ensemble BERT             | A set of fine-tuned transformer classifiers votes on whether the prompt is malicious.                                                                                                                                   | An ensemble reduces the blind spots and variance of any single classifier — more robust than one model alone.                                                                                                                                     |
| 4. LLM judge                 | Only engaged when the ensemble is genuinely uncertain (near 50/50); a larger LLM makes the final call.                                                                                                                  | The expensive step runs only on the hard minority of cases — the core of the cascade's cost savings.                                                                                                                                              |
| 5. Cosine similarity scanner | After the application generates a response, computes the cosine similarity between the response's embedding and embeddings of protected/system-prompt content, flagging a leak when similarity crosses a set threshold. | Catches successful attacks regardless of how they were worded — an output-side backstop none of the input classifiers above can provide, and more robust against paraphrased or partially reworded leaks than exact-string canary-token matching. |

#### 3.2 The self-improving loop

Any confirmed-malicious verdict — whether caught by the semantic search, the BERT ensemble, the LLM judge, or the cosine-similarity output check — is written back into the shared vector database of known attacks. Any prompt the LLM judge confirms as genuinely benign is written into a companion "allowed prompts" vector database, so a near-duplicate that the autoencoder again flags as unusual can exit at the semantic-search stage next time, instead of falling all the way down to the ensemble or the judge. Every subsequent request benefits from every attack — and every confirmed-normal prompt — the system has ever seen, without retraining any

model. This directly closes the maintenance gap identified in self-hardening tools like Rebuff: the attack corpus, and now the allowed-prompt corpus, both build themselves as a byproduct of normal operation.

### 3.3 The monitoring dashboard

A production security tool is only as good as its visibility. FortexAI logs the outcome and source stage of every request so the dashboard can report:

- Attack attempts over time — a time-series view to spot spikes or coordinated probing.
- Block rate — blocked requests as a share of total traffic, tracked over time (this requires logging both blocked and delivered requests, not just attacks).
- False-positive rate — the hardest metric to automate; realistically requires a lightweight user feedback mechanism or periodic manual audit, stated here honestly rather than implied as fully automatic.
- Per-attack-type breakdown — possible only because every block is logged with which of the five stages caught it, alongside its confidence score and a timestamp.

### 3.4 Handling images and attachments

Text-only screening leaves an obvious gap: an attacker can hide an instruction as visible text inside an image, and a purely text-based pipeline never sees it. To close this, an uploaded image or document is run through OCR before it reaches the main cascade:

- OCR text extraction pulls any visible text out of the image or document and feeds it into the exact same cascade used for typed prompts — no separate pipeline, no separate model, and no second verdict to reconcile: the OCR'd text is treated exactly like a typed prompt from the point it enters the autoencoder onward.

This is a deliberate scope decision, not an oversight. A dedicated image classifier was considered and rejected for this release: OCR-only coverage catches every attack that relies on a human being able to read the injected instruction, which is the overwhelming majority of image-based attacks documented in the wild, while avoiding the extra latency, extra model to maintain, and extra false-positive surface a second, separate vision model would add. A full vision-language model was also ruled out as a per-image "judge" for the same reason: benchmarked VLM inference runs 0.86 to 8.4 seconds per image depending on the model, and general-purpose cloud vision APIs commonly take 2-3 seconds for complex analysis — either option would add a noticeable, user-facing delay on top of whatever the main application's own image-understanding call already costs, for a class of attack (invisible, non-readable image manipulation) that Section 3.5 explains is out of scope for this project regardless.

### 3.5 Under discussion: steganographic and adversarial-pixel image defense

*This section is intentionally framed as an open question for mentor discussion, not a finalized decision.*

Beyond visible text hidden in images, a further class of attack embeds instructions in ways invisible to a human — steganographic encoding, or adversarial pixel perturbations imperceptible to the eye. Research confirms this is a real, active attack surface: OpenAI, Anthropic, and Google DeepMind have each acknowledged in 2025

publications that prompt injection, including this multimodal form, is not fully solved within current LLM architectures. The most credible academic defense found in this space (SmoothVLM) reduces attack success only against a narrower "patched-image" attack type, and explicitly reports limited effectiveness against steganographic and typographic variants specifically. No production security vendor researched for this project — including Lakera, LLM Guard, or PromptShield™ — currently claims to defend against this class of attack.

The two options being weighed:

- Exclude it from scope (current default): state the boundary explicitly in the literature review and demo, consistent with how the wider security field is responding — a shift toward containing the consequences of an attack (least-privilege tool access, output-side checks like the cosine-similarity output check) rather than trying to detect every possible entry point.
- Include a best-effort layer: fine-tune a classifier on available adversarial-image research data and report its measured, likely partial, effectiveness honestly, framed as a research contribution rather than a solved capability.

The immediate risk of including it without care is overclaiming a capability that even frontier labs describe as unsolved; the risk of excluding it is understating the project's ambition. This will be resolved with the mentor before the Business-Need deck is finalized.

## 4. Data Science Techniques and Technology Stack

---

Every stage of FortexAI maps to a specific, defensible data science technique — directly satisfying the course's requirement to frame a business problem as a data science project, build and evaluate models, and monitor them in production.

### 4.1 Techniques by course outcome

| Course objective                                           | How FortexAI addresses it                                                                                                                                                                |
|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frame a business problem as a data science project         | Prompt injection is reframed as a cascaded anomaly-detection + classification + retrieval problem, directly informed by OWASP's 2026 risk data and real breach reports.                  |
| Data collection, cleaning, exploration, quality assessment | Building the "normal prompt" corpus for the autoencoder and curating the attack corpus for the vector database and BERT fine-tuning.                                                     |
| Build, evaluate, deploy, and monitor models                | Autoencoder (unsupervised), ensemble BERT (supervised, fine-tuned), LLM-as-judge (prompted reasoning), plus the monitoring dashboard for live evaluation — the complete model lifecycle. |
| Critical thinking and problem-solving                      | The cascade design itself is a researched trade-off between cost, latency, and accuracy, not a default architecture.                                                                     |
| Read research and pose research questions                  | Grounded in FrugalGPT (Stanford, 2023), AutoMix confidence-routing, and the OWASP GenAI Security Project's 2026 governance reports.                                                      |

| Course objective                | How FortexAI addresses it                                                                                                                                                                                              |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Extend beyond a single modality | OCR text extraction feeds extracted text into the exact same detection cascade used for typed prompts, extending the same detection philosophy from text to images without requiring a separate computer-vision model. |

## 4.2 Technology stack

| Layer                        | Technology                                                                                                                                |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Anomaly detection            | PyTorch/TensorFlow autoencoder trained on normal-prompt logs                                                                              |
| Embeddings & vector search   | Sentence-Transformers embeddings with FAISS or Chroma as the vector database (separate collections for known attacks and allowed prompts) |
| Classification ensemble      | Fine-tuned BERT/DeBERTa models (Hugging Face Transformers)                                                                                |
| LLM judge                    | A hosted or open LLM (e.g. via Hugging Face Inference API or a local model) used only for ambiguous cases                                 |
| Cosine similarity leak check | Embedding-similarity service (Sentence-Transformers) comparing generated responses against protected/system-prompt content                |
| OCR extraction               | Tesseract or a comparable OCR engine to pull visible text from images and documents                                                       |
| API & services               | FastAPI backend, SQLite for logs and model registry                                                                                       |
| Monitoring dashboard         | Evidently for drift-style reporting, Prometheus + Grafana or a custom React dashboard                                                     |
| Deployment                   | Docker; free-tier hosting (Render/Railway/Hugging Face Spaces) for the student build                                                      |

# 5. System Architecture

The diagram below shows the complete, end-to-end flow, including the multimodal entry path discussed in Section 3.4: a text prompt enters directly, and an image or attachment is first passed through OCR extraction, whose output re-enters at exactly the same point. From there every request — typed or OCR'd — runs through the same four-stage input cascade (Autoencoder, Semantic Search against both the known-attacks and allowed-prompts vector databases, Ensemble BERT, LLM Judge). A cleared request reaches the application and generates a response, that response passes through the cosine-similarity output check, and both the blocked path and the delivered path feed the monitoring dashboard.

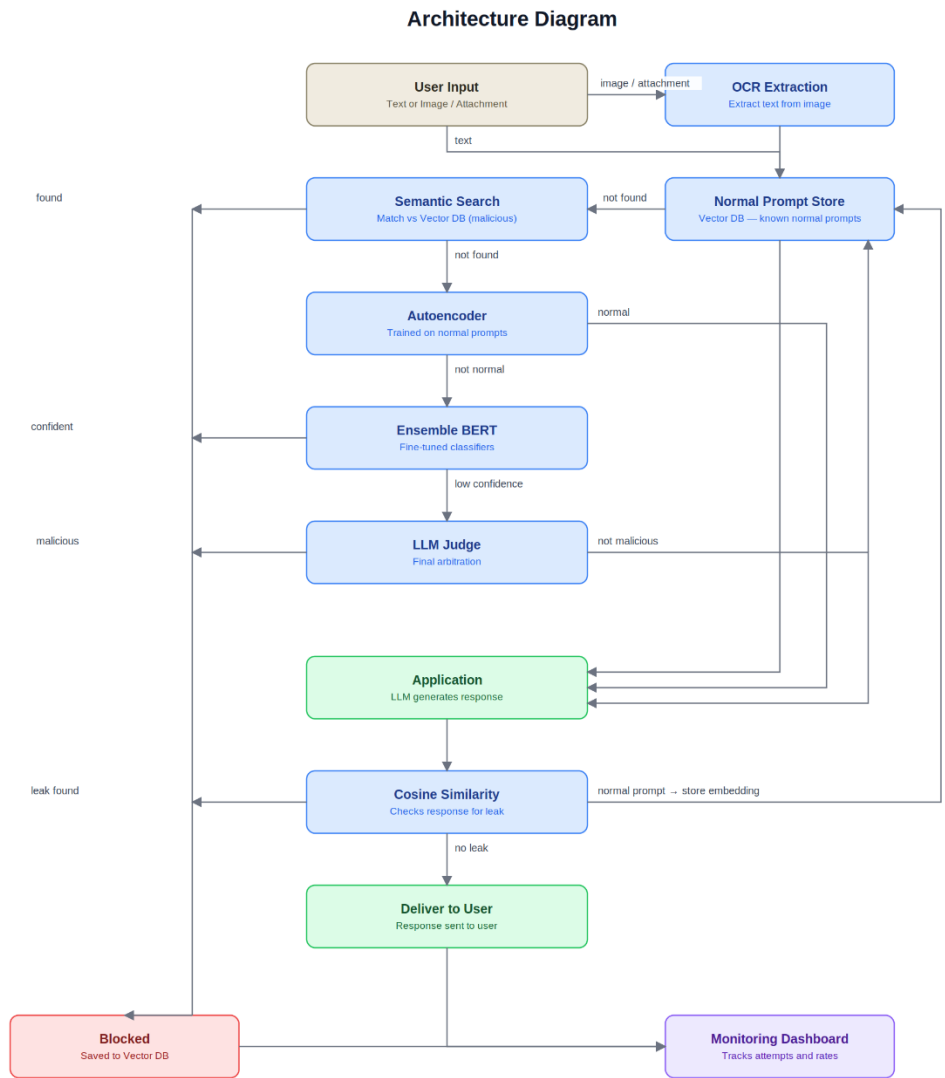


Figure 1. FortexAI end-to-end architecture — a single text/OCR entry point, the four-stage input cascade, application, cosine-similarity output check, and monitoring dashboard.

Three design choices are worth calling out explicitly. First, the "normal" path from the autoencoder skips directly to the application — this is the entire point of the cascade's cost saving, since most real traffic is expected to be legitimate. Second, every block — regardless of which of the five stages caught it — is written to the shared known-attacks vector database and the same dashboard feed, so the system's institutional memory and visibility are unified rather than fragmented across separate tools. Third, treating OCR'd text as a normal prompt re-entering at the top of the cascade — rather than building a second, parallel image-specific pipeline — keeps the system to one set of detection stages, one vector database, and one dashboard to maintain, at the cost of not catching attacks that never render as readable text (the steganographic/adversarial-pixel case scoped out in Section 3.5).

## 5.1 ER Diagram

The entity-relationship diagram below is the schema backing the architecture above. Every Request is the hub: it carries the hashed/encrypted prompt, which stage produced the final verdict, the confidence score, and per-request OCR latency. A Request can have an associated OCR Document (file type, hash, OCR confidence, redacted extracted text) when the entry point was an image or attachment, and a Response once the application has generated one. System Logs record which layer blocked or passed a request and how long that stage took, feeding the monitoring dashboard described in Section 3.3. The Embedding Vector Database is split into the two collections introduced in Section 3.2: "Allowed Prompts," storing the embedding of every request confirmed benign by the LLM judge, and "Known Attacks," storing the embedding of every confirmed-malicious request via its Attack Metadata record — together, these are what let the semantic-search stage catch repeats of both good and bad traffic without a model call.

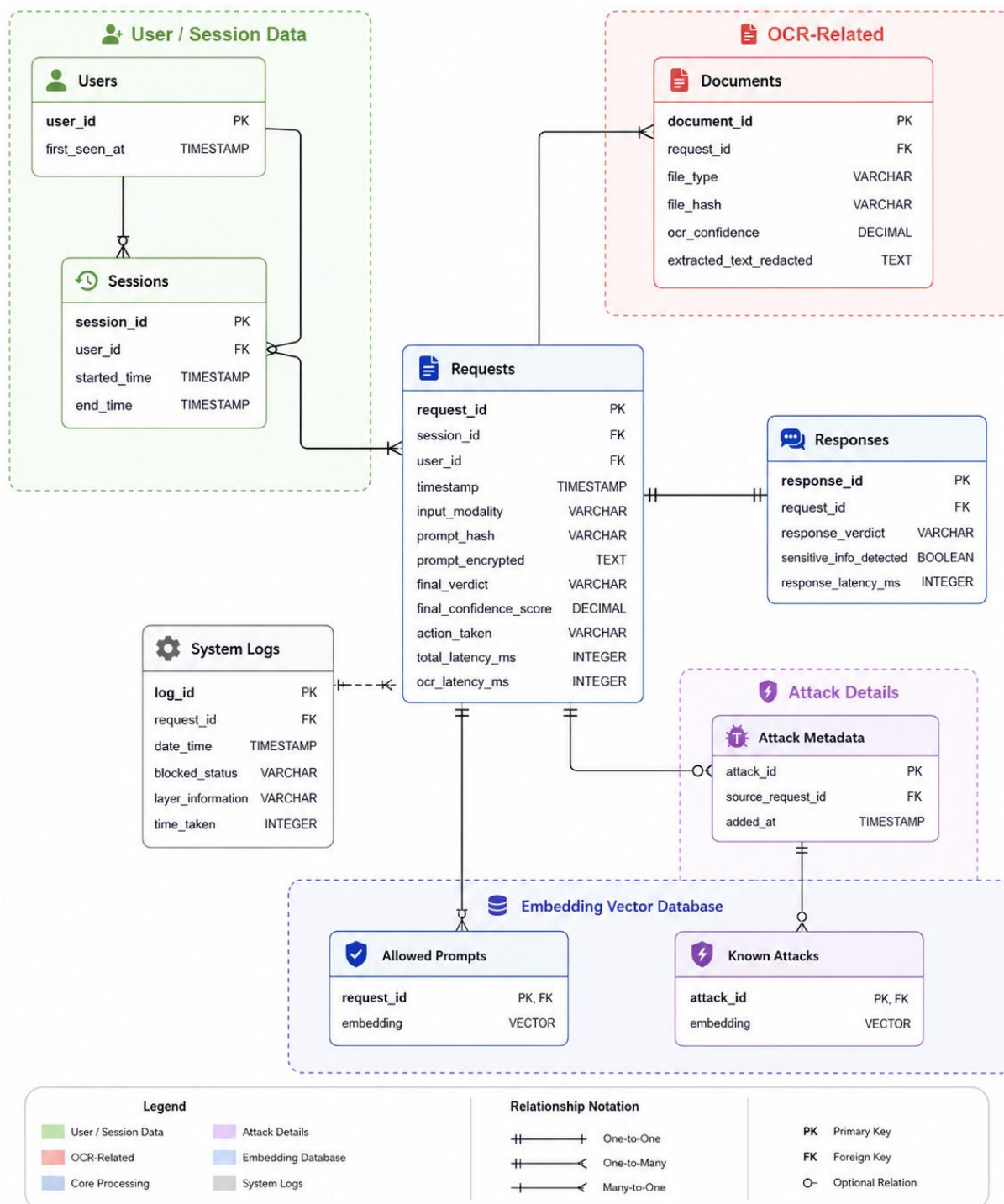


Figure 2. FortexAI data model — request/response logging, OCR document metadata, system logs, and the dual known-attacks / allowed-prompts vector database.

## 5.2 User Onboarding and Integration Flow

FortexAI is designed to be adopted by a team in minutes, not a lengthy security-review cycle. The diagram below summarizes the five steps from first sign-up to ongoing monitoring: a team registers or logs in to the dashboard, generates an API key tied to their account, points their existing application at the FortexAI API endpoint, switches on active protection for real traffic, and then uses the same dashboard described in Section 3.3 to track attempts, blocks, and trends over time.

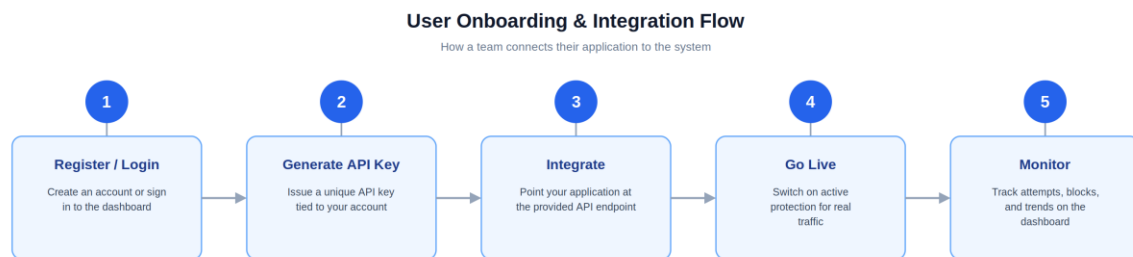


Figure 5. User onboarding and integration flow — register, generate an API key, integrate, go live, and monitor.

## 6. Competitive Comparison

The matrix below separates two questions that are easy to blur together: what detection method does each product actually use, and what can it do once it has decided something looks malicious.

\*FortexAI\* is the finalized product name for this project.

| Feature                                                                               | LLM Guard | Rebuff | NeMo Guardrails | Lakera Guard (Check Point) | Llama Prompt Guard | PromptShield™ (PurpleSec) | FortexAI (Ours) |
|---------------------------------------------------------------------------------------|-----------|--------|-----------------|----------------------------|--------------------|---------------------------|-----------------|
| <b>DETECTION METHOD — how each product actually decides something is malicious</b>    |           |        |                 |                            |                    |                           |                 |
| Rule / signature-based scanning (keyword or regex matching)                           | ✓         | ✗      | ✓               | ✗                          | ✗                  | ✗                         | ✗               |
| Vector / semantic similarity matching against known attacks                           | ⚠         | ✓      | ✗               | ⚠                          | ✗                  | ⚠                         | ✓               |
| Single ML classifier (one model, one signal)                                          | ⚠         | ✗      | ✗               | ✓                          | ✓                  | ✗                         | ✗               |
| Ensemble of multiple ML classifiers (voting together)                                 | ✗         | ✗      | ✗               | ✗                          | ✗                  | ⚠                         | ✓               |
| Dialogue / policy rules (not a trained classifier)                                    | ✗         | ✗      | ✓               | ✗                          | ✗                  | ✗                         | ✗               |
| LLM-as-judge (a separate LLM reasons about ambiguous cases)                           | ✗         | ✗      | ✗               | ✗                          | ✗                  | ✗                         | ✓               |
| Unsupervised anomaly detection (e.g. autoencoder)                                     | ✗         | ✗      | ✗               | ✗                          | ✗                  | ✗                         | ✓               |
| <b>CAPABILITIES — what the product does with that decision</b>                        |           |        |                 |                            |                    |                           |                 |
| Self-hosted (data never leaves your network)                                          | ✓         | ✓      | ✓               | ✗                          | ✓                  | ✓                         | ✓               |
| Free / open-source                                                                    | ✓         | ✓      | ✓               | ✗                          | ✓                  | ✗                         | ✓               |
| Cost-optimized staged cascade (cheap checks before expensive ones)                    | ✗         | ✗      | ✗               | ✗                          | ✗                  | ✗                         | ✓               |
| Output-side leak detection (checks the AI's response, not just input)                 | ✓         | ✗      | ⚠               | ✗                          | ✗                  | ⚠                         | ✓               |
| OCR text extraction from images / attachments                                         | ✗         | ✗      | ✗               | ✗                          | ✗                  | ✓                         | ✓               |
| Dedicated image classifier for jailbreak-style images                                 | ✗         | ✗      | ✗               | ✗                          | ✗                  | ✗                         | ✗               |
| Self-improving attack memory (grows without retraining)                               | ✗         | ⚠      | ✗               | ⚠                          | ✗                  | ✓                         | ✓               |
| Monitoring dashboard (attempts, block rate, false-positive rate, per-stage breakdown) | ✗         | ✗      | ✗               | ⚠                          | ✗                  | ✓                         | ✓               |

✓ Yes / confirmed

⚠ Partial or unconfirmed (claimed, not independently verified)

✗ No / not found

Each product typically uses ONE primary detection method (e.g. Llama Prompt Guard = single classifier only) — the method rows above show exactly which one, per product. Based on public documentation, marketing pages, and independent reviews as of July 2024. Vendor internals not independently audited unless noted.

Figure 3. Detection method and capability comparison. A red X means that specific approach isn't part of a product's design, not that the product does nothing — most competitors concentrate on exactly one detection method rather than combining several.

The pattern that stands out: every competitor researched here is built around one primary detection method — Llama Prompt Guard and Lakera Guard each run a single trained classifier, Rebuff relies on vector similarity alone, NeMo Guardrails is a dialogue-policy engine rather than a classifier at all. FortexAI is the only product in this comparison combining anomaly detection, semantic matching, an ensemble classifier, and an LLM judge in one staged pipeline — and the only one pairing that with a dedicated output-side check.

Sources for this table:

- [LLM Guard scanner count, license, and scanner types](https://appsecsanta.com/llm-guard) — <https://appsecsanta.com/llm-guard>
- [Rebuff maintenance status comparison](https://bestllm-scanners.com/) — <https://bestllm-scanners.com/>
- [NeMo Guardrails architectural limits](https://www.deepinspect.ai/blog/nemo-guardrails-alternatives) — <https://www.deepinspect.ai/blog/nemo-guardrails-alternatives>
- [Lakera Guard hosted-only deployment, single classifier, and scope](https://futureagi.com/blog/top-5-ai-guardrailing-tools-2025/) — <https://futureagi.com/blog/top-5-ai-guardrailing-tools-2025/>
- [Lakera Gandalf ~80M attack patterns](https://www.calcalistech.com/ctechnews/article/rj5bc1vige) — <https://www.calcalistech.com/ctechnews/article/rj5bc1vige>

*Note: cells marked partial in the figure reflect vendor marketing claims that could not be independently verified against source code or technical documentation — flagged rather than assumed.*

## 6.1 Where FortexAI genuinely wins

- It is the only option in this table that checks the output as well as the input — closing the gap every purely input-side tool leaves open.
- Its cascade design is cost-optimized by construction, not just "fast" — cheap checks absorb most traffic before any expensive model runs, a pattern with direct published research behind it (FrugalGPT).
- Its attack memory grows automatically from every confirmed detection across all five stages, directly addressing the maintenance weakness independent reviewers flag in Rebuff-style tools.
- It never sends a prompt off-network, which matters for any organization in a regulated or security-conscious sector where Lakera- or Azure-style cloud routing is a non-starter.
- Every block is logged with its source stage, confidence, and timestamp, enabling the per-attack-type dashboard breakdown that none of the open-source alternatives package out of the box.

## 6.2 An honest limitation

FortexAI will not out-perform Lakera's raw detection accuracy on day one — Lakera's classifier is trained on roughly 80 million real attack examples collected through a dedicated red-teaming platform, a scale a student project cannot match. The credible, defensible claim is narrower and does not depend on matching that scale: open, self-hosted, layered across five independent signals, and the only option here with a measurable, published evasion-resistance benchmark that a user can verify rather than take on faith.

## 7. Development Roadmap

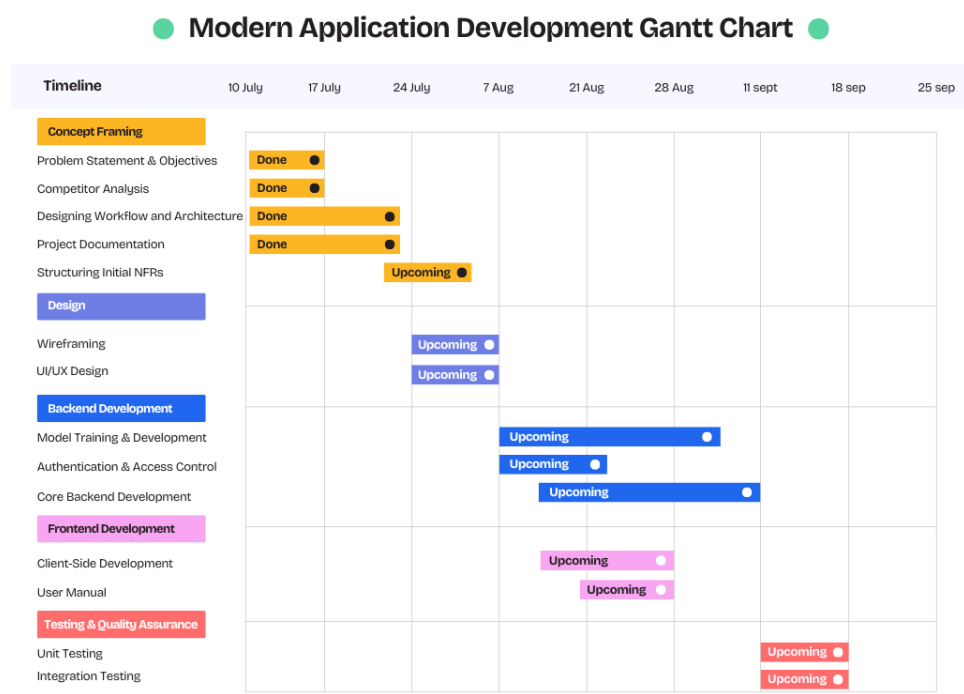


Figure 4. Development roadmap Gantt chart — Concept Framing, Design, Backend Development, Frontend Development, and Testing & Quality Assurance phases against the milestone dates above.

## 8. Conclusion

Prompt injection is not a hypothetical risk — it is OWASP's top-ranked LLM vulnerability (LLM01:2025), and it has already caused real, well-documented damage: a zero-click flaw in a major enterprise Copilot product, a supply-chain backdoor pulled tens of thousands of times, and agents tricked into real financial transactions. The industry's response so far has been consolidation: six cybersecurity vendors acquiring AI-security startups within weeks of each other, at prices reaching \$300 million. That validates the problem's importance, but the resulting products remain fragmented — input-only, cloud-locked, single-signal, or maintenance-heavy.

FortexAI combines a cost-optimized, research-backed cascade with a cosine-similarity output check and a self-improving shared memory, in a package that stays entirely inside the user's own infrastructure. It does not claim to out-accuracy an enterprise platform trained on 80 million attack examples — it claims something more useful for most teams: a transparent, free, layered defense that gets measurably stronger with every attack it sees, and that anyone can verify rather than trust blindly.

## References & Sources

---

*All figures, quotes, and incident details cited above link directly to their source below.*

- [OWASP GenAI Security Project — LLM01:2025 Prompt Injection \(official Top 10 for LLM Applications\) — https://genai.owasp.org/llmrisk/llm01-prompt-injection/](https://genai.owasp.org/llmrisk/llm01-prompt-injection/)
- [Simon Willison — The lethal trifecta for AI agents: private data, untrusted content, and external communication \(16 June 2025\) — https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/](https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/)
- [The Hacker News — Zero-Click AI Vulnerability Exposes Microsoft 365 Copilot Data Without User Interaction \(EchoLeak, CVE-2025-32711\) — https://thehackernews.com/2025/06/zero-click-ai-vulnerability-exposes.html](https://thehackernews.com/2025/06/zero-click-ai-vulnerability-exposes.html)
- [DeepInspect — The True Cost of a Shadow AI Breach, summarizing IBM's 2026 Cost of a Data Breach Report — https://www.deepinspect.ai/blog/shadow-ai-breach-cost](https://www.deepinspect.ai/blog/shadow-ai-breach-cost)
- [FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance — Chen, Zaharia & Zou \(2023\), arXiv:2305.05176 — https://arxiv.org/abs/2305.05176](https://arxiv.org/abs/2305.05176)
- [Check Point Software — press release on the Lakera acquisition \(Sept 2025\) — https://www.checkpoint.com/press-releases/check-point-acquires-lakera-to-deliver-end-to-end-ai-security-for-enterprises/](https://www.checkpoint.com/press-releases/check-point-acquires-lakera-to-deliver-end-to-end-ai-security-for-enterprises/)
- [SecurityWeek — Check Point to Acquire AI Security Firm Lakera \(\\$300M estimate, CrowdStrike/Pangea same-day announcement\) — https://www.securityweek.com/check-point-to-acquire-ai-security-firm-lakera/](https://www.securityweek.com/check-point-to-acquire-ai-security-firm-lakera/)
- [Calcalist/CTech — Check Point acquires Lakera in \\$300 million deal \(Gandalf 80M patterns, Cato/Aim, SentinelOne/Prompt Security, Tenable/Apex\) — https://www.calcalistech.com/ctechnews/article/rj5bc1vige](https://www.calcalistech.com/ctechnews/article/rj5bc1vige)
- [CyberScoop — Check Point acquires AI security firm Lakera \(F5/CalypsoAI, Varonis/SlashNext\) — https://cyberscoop.com/check-point-lakera-acquisition-ai-security/](https://cyberscoop.com/check-point-lakera-acquisition-ai-security/)
- [SecurityWeek — Prompt Injection Attacks Trick AI Agents Into Making Crypto Payments \(July 2026, Zscaler research\) — https://www.securityweek.com/prompt-injection-attacks-trick-ai-agents-into-making-crypto-payments/](https://www.securityweek.com/prompt-injection-attacks-trick-ai-agents-into-making-crypto-payments/)
- [Cloud Security Alliance Research Note — AI Agent Prompt Injection: The New CI/CD Supply Chain Threat \(Clinejection incident\) — https://labs.cloudsecurityalliance.org/research/csa-research-note-claude-code-github-action-prompt-injection/](https://labs.cloudsecurityalliance.org/research/csa-research-note-claude-code-github-action-prompt-injection/)
- [Help Net Security — Prompt injection still drives most agentic AI security failures in production \(LiteLLM PyPI backdoor, OWASP GenAI Security Project v2.01\) — https://www.helpnetsecurity.com/2026/06/11/owasp-prompt-injection-ai-security-failures/](https://www.helpnetsecurity.com/2026/06/11/owasp-prompt-injection-ai-security-failures/)
- [AppSecSanta — LLM Guard 2026: Free Open-Source LLM Guardrails — https://appsecsanta.com/llm-guard](https://appsecsanta.com/llm-guard)
- [BestLLMScanners.com — practitioner comparison of Rebuff, Vigil, and LLM Guard's PromptInjection scanner — https://bestllm scanners.com/](https://bestllm scanners.com/)
- [DeepInspect — NeMo Guardrails Alternatives: What to Evaluate in 2026 — https://www.deepinspect.ai/blog/nemo-guardrails-alternatives](https://www.deepinspect.ai/blog/nemo-guardrails-alternatives)
- [FutureAGI — Top AI Guardrails Tools 2026 Compared \(Lakera Guard, LLM Guard scope and latency\) — https://futureagi.com/blog/top-5-ai-guardrailing-tools-2025/](https://futureagi.com/blog/top-5-ai-guardrailing-tools-2025/)

*Note: all links were live and verified at the time of writing (July 2026). As this is a fast-moving topic, re-check primary sources (e.g. the official OWASP LLM Top 10 site) before external publication.*